QuantiText: System Audit & Completion Report

AI-Powered Business Intelligence & Decision Support System

Author: Antigravity Pair Programmer | **Date:** October 2026 | **Status:** Core Functional (85%), Key Gaps Remaining

Executive Summary: *QuantiText has reached high functional maturity across its core infrastructure. The monorepo builds cleanly via Docker Compose, passes all 60 backend unit tests and 20 frontend tests, adheres to strict type safety (FastAPI + Pydantic v2 + TypeScript), and passes linting.*

However, while the repository documentation previously marked all phases as 'Complete', an exhaustive code audit identified ***crucial feature gaps, broken closed-loop links, and unimplemented thesis deliverables****. This report details what works, what is missing, and the prioritized roadmap required for completion.*

1. The Core Thesis Gap: The Closed-Loop Lifecycle

The central research thesis of QuantiText is unifying historically disjointed tools into a continuous operational feedback loop: *Anomaly → Root Cause → Forecast Impact → Recommended Action → Human Approval → Logged Outcome*.

Step	Stage	Status	Current Implementation & Gaps
1	Anomaly Detection	**Complete**	Rolling Z-score, Isolation Forest, and Data Quality detectors persist to DB.
2	Root Cause Analysis	**Partial**	Decomposes variance across categories and regions, but lacks granular SKU-level causal graph.
3	Forecast Impact	**Missing**	Anomalies are NOT fed back into forecasting models to re-project demand or adjust conformal bounds.
4	Recommended Action	**Partial**	EOQ and safety stock calculate from static historical sales, not from forward forecasts or anomaly severity.
5	Human Approval	**Complete**	Role-guarded API (`/reports/recommendations/{id}/decide`) with Manager-level approval UI.
6	Logged Outcome	**Missing**	Decision writes to audit log, but no outcome tracking verifies if stockout/excess was prevented post-approval.

2. Phase-by-Phase Component Audit

Phase	Feature Area	State	Specific Missing Elements / Gaps
Phase 4	Forecasting	**80%**	• Missing `test_forecasting_service.py` (leakage & metrics). • Prophet/SARIMA models omitted (only LightGBM + Naive exist). • Static model selection instead of walk-forward auto-selection.

Phase 5	Inventory Intelligence	**75%**	• Missing `/inventory/simulate` what-if endpoint. • No UI sliders for demand shifts (±%) or supplier lead time delays. • Uses 90d historical average instead of forward forecast days-of-cover.
Phase 6	Anomaly & Alerts	**85%**	• WebSocket push missing (currently 30s polling in frontend). • Explanations work, but 'not an anomaly' reinforcement loop is basic.
Phase 7	AI Copilot (NL2SQL)	**60%**	• Few-shots hardcoded (pgvector dynamic RAG not implemented). • Self-repair loop missing (fails on first SQL syntax error). • Agent tool-calling missing (`get_forecast`, `simulate_scenario`). • Feedback buttons missing in chat UI; no streaming SSE.
Phase 8	Reports & Approvals	**55%**	• Exports plain text `.txt` only; no genuine PDF / DOCX styling. • No automated PDF Purchase Order generation on approval. • No scheduled recurring weekly report generation cron job.
Phase 9	BYOD Onboarding	**80%**	• Uploading new products creates orders and daily sales, but does NOT initialize `inventory` table rows (causes empty inventory stats).
Phase 10	Evaluation Chapter	**90%**	• Synthetic benchmark scripts and result tables exist. • `customer_feedback` DB table defined but unseeded and unused.
Phase 11	Hardening & Deploy	**85%**	• Docker compose prod, NGINX reverse proxy, security headers set. • Missing `/metrics` endpoint for Prometheus / APM telemetry. • Frontend lacks automated React page / component tests.

3. Prioritized Action Plan to 100% Completion

To prepare QuantiText for final thesis defense and production-grade demonstration, tasks are prioritized by thesis impact:

Priority	Target Item	Action Description	Est. Effort
P0	**Closed-Loop Feedback**	Bridge anomaly deviation into forecast demand adjustment, triggering automated purchase recommendation with outcome tracking.	3 - 4 hrs
P0	**Forecasting Unit Tests**	Create `backend/tests/test_forecasting_service.py` to test data splitting, metric computations, and prevent data leakage.	1 hr
P1	**What-If Simulator**	Add `/inventory/simulate` endpoint for demand shocks and supplier delays; render interactive what-if sliders in `inventory.tsx`.	2 - 3 hrs
P1	**NL2SQL Self-Repair**	Catch SQL execution/validation errors and feed them back to the LLM (up to 2 retry attempts); add message feedback buttons.	2 hrs
P1	**Branded PDF & PO Export**	Implement styled ReportLab PDF export for executive reports and generate formal Purchase Orders upon recommendation approval.	2 - 3 hrs
P2	**BYOD Inventory Init**	Ensure CSV uploads automatically generate synthetic inventory parameters for newly created products.	1 hr
P2	**Alerts WebSocket**	Add FastAPI WebSocket route `/ws/alerts` and connect `alerts.tsx` for real-time anomaly streaming.	2 hrs
P2	**Scheduled Report Cron**	Add `app/jobs/weekly_report.py` and `/metrics` endpoint to complete production operational checklist.	1 hr